CODA: CASCADED ONLINE DISCONTINUITY-AWARE ALIGNMENT FOR REAL-TIME SCORE FOLLOWING

Yining Yang[1*] **Ruogu Chen**[2*] **Jie Han**[2]
[1] Don Wright Faculty of Music, Western University, Canada
[2] Department of Electrical and Computer Engineering, University of Alberta, Canada
yyang524@uwo.ca, ruogu@ualberta.ca, jhan8@ualberta.ca
*Equal contribution

ABSTRACT

Real-time score following from sheet images remains challenging because the model must process streaming audio while resolving highly repetitive visual patterns under strict latency constraints. Recent image-based methods have attempted to use multi-resolution prediction by simultaneously predicting the positions of the active system, bar, and note. However, their predictions across these different levels of notation are independent, which makes the predictions unstable and introduces unnecessary extra search space for bar- and note-level predictions. Most existing methods also lack mechanisms to recover from score discontinuities, such as repeats, da capo (D.C.), or coda jumps. This paper proposes CODA, to the best of our knowledge, the first real-time score following system that addresses both gaps. CODA explicitly exploits the cascaded structure of music scores: it first selects the active system, then the active bar within it, and finally the active note within the selected bar. This enforces prediction consistency across resolutions. A silence-driven break mode enables recovery from arbitrary score discontinuities without requiring knowledge of the repeat structure. Evaluated on the Multimodal Sheet Music Dataset (MSMD) piano benchmarks, CODA achieves state-of-the-art tracking accuracy and discontinuity-recovery performance under real-time throughput. Code is available at https://github.com/ValleyC/CODA.

1. INTRODUCTION

Real-time score following is the process of aligning a live performance with the corresponding music score as the music proceeds in real time. It is fundamental for downstream applications, including automatic accompaniment, automatic page-turning, synchronized score display, and many other interactive music systems [1, 2]. Classical approaches use online Dynamic Time Warping (DTW) or probabilistic state models to achieve real-time alignment [3–5]. However, these methods require a symbolic score such as MIDI or MusicXML, which is not always available.

Image-based score following has emerged over the past decade by working directly on sheet images. Early work classified positions into coarse staff-level buckets on small local image crops [6], limiting both spatial resolution and the field of view. Reinforcement-learning methods widened the view to score strips but still operated on narrow windows, making re-engagement difficult when the tracker drifted off track [7, 8]. Full-page segmentation addressed the field-of-view limitation by predicting a pixel-level heatmap over the entire page [9], but only localized at the note level without explicit system or bar tracking. The most recent of these image-based methods proposed multi-resolution detection with You Only Look Once (YOLO)-based object detection frameworks [10]. This work predicts the positions of systems, bars, and notes simultaneously using three independent output heads. However, there are no constraints or cascaded structures among these predictions. Nothing enforces that the predicted note lies within the predicted bar, or that the predicted bar lies within the predicted system. The model implicitly learns structural consistency, and disagreements among the three prediction levels are often observed.

This independence causes another problem. Because each head independently predicts over the full page, the bar and note heads must search all candidates on the page, increasing the likelihood of errors and instabilities. A cascaded structure would narrow the search at each stage: once the system is selected, only bars within that system are candidates; once a bar is selected, the candidate note positions are confined to that bar. More fundamentally, this method frames score tracking as an object detection problem. It detects from scratch at every audio frame, predicting bounding boxes as if the system and bar locations were unknown. However, in music performances, the score is static, and all candidate positions are known in advance. Detection is therefore unnecessary. The real task is to select which of the known candidates is currently being played. This formulation is feasible whenever score layout information is available.

Additionally, existing image-based methods lack mechanisms to recover from score discontinuities such as repeats, da capo (D.C.), or coda jumps, even though such

events are common in practice and have long been addressed in symbolic score following [5, 11, 12]. In our analysis of the MSMD test set, 66 out of 94 pieces contain written repeat structures, producing 131 jumps in standard performance order. Despite this prevalence, no existing real-time image-based score following method includes an explicit mechanism for handling such events during online tracking.

This paper proposes CODA, a cascaded online alignment framework that addresses these gaps. The key observation is that on a fixed score page, all system and bar locations are already known. The tracking problem is therefore reformulated as a selection task among known candidates. CODA follows the cascaded structure of music scores directly: it first selects the active system among all systems, then the active bar within that selected system, and finally the note position inside the selected bar. This formulation naturally enforces geometric consistency and narrows the search space at each stage. To handle score discontinuities, CODA models all jumps as arbitrary position changes preceded by silence, enabling a single recovery mechanism that generalizes across repeats, D.C., coda jumps, and performer errors without requiring knowledge of the score's repeat structure. The major contributions are:

- A cascaded selection-and-regression formulation that enforces geometric consistency across system, bar, and note predictions while narrowing the search space at each stage.
- A causal streaming architecture combining Mamba audio encoding, FiLM conditioning, cross-attention, and beam search with learned temporal priors.
- A silence-driven jump recovery mechanism for arbitrary score discontinuities, including repeats, D.C., and coda jumps, without requiring prior knowledge of the score's repeat structure.
- A repeat-aware jump test benchmark with both annotated and random music discontinuities for all 94 pieces in the MSMD test set, providing the first standardized evaluation protocol for discontinuity handling in image-based score following.

2. RELATED WORK

2.1 Symbolic Score Following

Classical real-time score following relies on symbolic scores such as MIDI or MusicXML. Online DTW [3] provides a straightforward baseline by incrementally matching audio frames to score events. Probabilistic state-space models offer richer temporal modeling: Cont [4] introduced a coupled duration-focused architecture, while Arzt and Widmer [13] addressed robustness under arbitrary performer deviations using multi-agent tracking. Particle-filter methods such as that of Korzeniowski et al. [14] extended the probabilistic framework to handle tempo changes and rests. Jiang and Raphael [15] proposed a switching state-space model that tracks hidden tempo changes, improving alignment stability under expressive performance variation. Nakamura et al. [5] modeled repeats and skips with an explicit transition structure. Their analysis further revealed that the majority of real-world score jumps are preceded by short silent breaks, an observation that motivates the jump recovery design in Section 3.6. More recently, Peter et al. [16] combined real-time piano transcription with symbol-level tracking, achieving strong results by leveraging automatic music transcription as a front end. Matchmaker [12] provides a systematic evaluation framework for symbolic methods, though it does not cover image-based trackers.

2.2 Image-Based Score Following

Dorfer et al. [6] first showed that a multimodal convolutional neural network (CNN) can follow sheet music without symbolic input, classifying position into discrete staff-level buckets on small image crops. Reinforcement-learning formulations then framed the task as agent navigation over score strips [7, 8]. Henkel et al. [9] moved to full-page processing via audio-conditioned U-Net segmentation, predicting a pixel mask whose center of mass gives the note position. CYOLO-SB [10] extended this to multi-resolution detection: three independent YOLO-style heads predict note, bar, and system bounding boxes from feature maps conditioned via Feature-wise Linear Modulation (FiLM) [17]. However, the three heads predict in parallel without cross-level constraints, predictions at each frame are decoded without temporal consistency constraints across frames, and no recovery mechanism exists for when tracking is lost.

2.3 Handling Repeats and Jumps

Score discontinuities, including repeats, D.C., and coda jumps, pose a long-standing challenge for score following systems. In the symbolic domain, Nakamura et al. [5] handle arbitrary repeats and skips through explicit state transitions. For offline audio-to-sheet-image synchronization, Shan and Tsai [18] proposed hierarchical DTW over learned audio and image features to handle repeats and jumps without requiring prior knowledge of jump locations. Morsi and Serra [11] identified repeat handling as one of the key bottlenecks in audio-to-score alignment research. However, no existing real-time image-based score following method includes an explicit mechanism for recovering from score discontinuities during online tracking.

3. METHOD

3.1 Problem Formulation

CODA operates on the entire score page. Layout metadata provides a finite set of systems and bars, along with their bounding boxes, for each page. The model selects among candidate systems and bars on the current page.

At each audio frame t, let $h_t = (x_{\leq t}, I)$ denote the causal audio history and the score image for the current page. The model predicts three quantities: the active system index s_t, the active bar index b_t within that system,

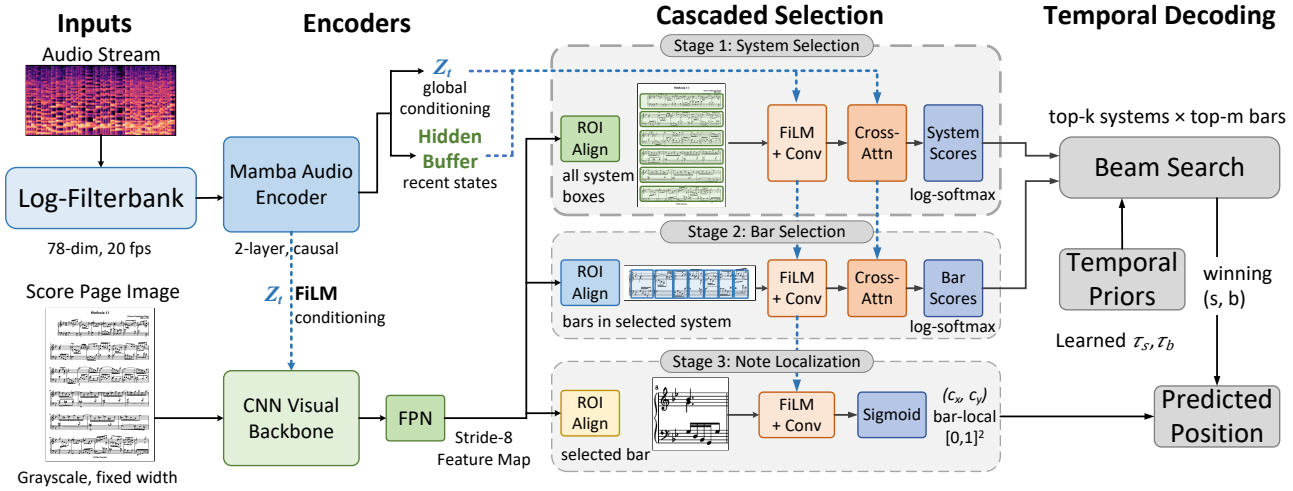

Figure 1: Overview of the CODA architecture. A causal Mamba audio encoder and a shared CNN visual backbone feed into three cascaded stages: system selection, bar selection, and note localization. Beam search with learned temporal priors decodes the cascade over time.

and a continuous note position $u_t = (c_x, c_y)$ expressed in bar-local coordinates. Rather than predicting all three independently, CODA factorizes the joint distribution as

$$p(s_t, b_t, u_t \mid h_t) = p(s_t \mid h_t) \cdot p(b_t \mid s_t, h_t) \cdot p(u_t \mid b_t, s_t, h_t). \quad (1)$$

This cascaded factorization enforces valid geometric structure by construction: the predicted bar always lies within the predicted system, and the predicted note always lies within the predicted bar.

3.2 Model Architecture

Figure 1 illustrates CODA's overall architecture. CODA has two input streams: audio and visual. The audio stream is converted into a 78-dimensional log-filterbank at 20 frames per second. A two-layer causal Mamba encoder [19] processes each frame. Its recurrent state compactly accumulates the entire audio history up to frame t, yielding a conditioning vector z_t. In parallel, a sliding window H_t of the most recent L per-frame outputs is maintained. z_t provides conditioning for the visual feature map via FiLM, while H_t serves as the key and value source for cross-attention between candidate visual regions and the recent audio history (Section 3.3).

The visual stream takes the score page image as input. The score page image is processed by a CNN backbone with FiLM conditioning driven by z_t at deeper stages. The output of the CNN backbone is further processed by a Feature Pyramid Network (FPN) that produces a stride-8 feature map F (i.e., at one-eighth the spatial resolution of the input). This feature map is shared across all three cascade stages (Section 3.3).

3.3 Cascaded Selection

Given the shared stride-8 feature map F from the visual input stream and the Mamba encoder outputs from the audio input stream, CODA applies three cascaded stages to localize the active position on the score page. Each stage narrows the spatial scope before proceeding to the next stage. All three stages share a common processing pipeline, with stage-specific variations detailed below. Figure 2 illustrates the output on a real score page: the selected system (green), bar (blue), and note position (red) are each geometrically contained within the previous level.

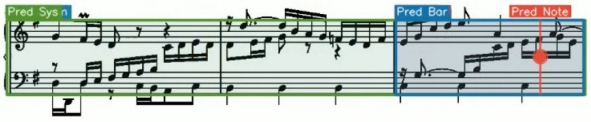

Figure 2: CODA tracking output on an MSMD score page.

Recall that $H_t \in \mathbb{R}^{L \times d}$ is the sliding window of the most recent L Mamba outputs, with d as the hidden dimension of the encoder. For each candidate region r (a system or bar bounding box from the score layout metadata), the pipeline proceeds as follows.

First, Region of Interest (ROI) Align [20] extracts a fixed-size feature patch $f_r = \mathrm{ROIAlign}(F, r)$ from the stride-8 feature map. The extracted features are modulated by the audio vector z_t via FiLM as $\tilde{f}_r = \gamma(z_t) \odot f_r + \beta(z_t)$, where γ and β are learned linear projections and $\odot$ denotes element-wise multiplication. Convolutional layers further process the conditioned features: $\hat{f}_r = \mathrm{Conv}(\tilde{f}_r)$. In the system and bar stages, the convolved features are refined by scaled dot-product cross-attention [21] over the audio buffer: $\bar{f}_r = \mathrm{softmax}\left(\mathrm{flat}(\hat{f}_r)\,(W_a H_t)^\top / \sqrt{C}\right) W_a H_t$, where queries are formed by spatially flattening $\hat{f}_r$, keys and values are shared projections of H_t, and W_a is a learned linear projection onto the C-dimensional attention space shared with $\hat{f}_r$. This cross-attention lets each candidate region attend to fine-grained temporal patterns in the recent audio history.

The system stage applies the full pipeline above (ROI

Align, FiLM, convolution, cross-attention) to every system candidate on the page, producing $\log p(s_t \mid h_t)$ via adaptive average pooling, a linear classifier, and softmax normalization. The bar stage applies the same pipeline with independent parameters to the bars within the selected system, yielding $\log p(b_t \mid s_t, h_t)$. The note stage applies only ROI Align, FiLM, and convolution (no cross-attention at this stage) on the selected bar and regresses bar-local coordinates $u_t = (c_x, c_y) \in [0, 1]^2$ via sigmoid activation.

3.4 Beam Search and Temporal Priors

At each audio frame, the cascaded selection described above produces a log-probability over systems and another log-probability over bars within each system. To decode these scores over time, CODA uses beam search combined with learnable temporal priors.

At each frame t, the model first scores every system on the page and retains the top-k candidates. For each candidate system, the bars within that system are scored, and the top-m candidates are retained per system. The actual values of k and m are specified in Section 4.1. The note head is evaluated only on the winning system–bar pair after the beam has been resolved. This two-level pruning keeps inference efficient: the full system distribution is computed once per frame, but bar-level and note-level computation is restricted to the $k \times m$ beam.

The beam ranking combines model confidence with learned temporal priors that encode transition preferences between consecutive frames. Specifically, the composite score for a system–bar hypothesis (s, b) at frame t is

$$\begin{aligned}\text{score}_t(s, b) = \; & \log p(s_t \mid h_t) + \tau_s(s_t, s_{t-1}) \\ & + \log p(b_t \mid s_t, h_t) + \tau_b(b_t, b_{t-1}),\end{aligned} \tag{2}$$

where τ_s and τ_b are page-local transition penalties for systems and bars, respectively. These priors are implemented as learnable parameters that are trained end-to-end with the rest of the model. They are clamped to remain non-positive, with the stay transition (i.e., $s_t = s_{t-1}$ or $b_t = b_{t-1}$) fixed at zero. This parameterization biases the tracker toward smooth temporal progression by encouraging smooth transitions and penalizing large jumps. However, the penalties are bounded rather than hard-coded, so the model can still override them when the audio strongly indicates that a real jump is occurring. The jump detection and recovery mechanism will be covered in Section 3.6.

The hypothesis with the highest composite score determines the predicted system $\hat{s}_t$ and bar $\hat{b}_t$. The note head then regresses the note position $\hat{u}_t$ within the selected bar, and the final output is the triplet $(\hat{s}_t, \hat{b}_t, \hat{u}_t)$.

3.5 Training

Because CODA is cascaded, the bar and note stages only consider candidates within the selected system. This cascaded dependency means that if the system prediction is wrong, all downstream predictions will also be wrong. If the model is trained exclusively with ground-truth system labels, it never learns to handle incorrect system inputs, creating a mismatch between training and inference.

To address this, training proceeds in two phases using scheduled sampling [22]. In the first phase, the bar and note stages always receive the ground-truth system labels from training data. This allows the bar head to focus on discriminating among bar candidates under ideal system selections. In the second phase, starting from the first-phase checkpoint, the model's own system prediction is gradually mixed in. At each training step in the second phase, the ground-truth system is replaced by the model's self-predicted system with probability p_{pred}, which increases linearly from 0 to a maximum value $p_{\max}$ over training. When the predicted system is wrong and does not contain the ground-truth bar, the bar and note training losses for that frame are masked because there is no meaningful supervision target. The system loss remains active, so the system head continues to receive corrective gradients. This two-phase curriculum training schedule progressively closes the gap between training and inference conditions.

The system and bar stages are trained with cross-entropy loss over their respective candidate sets, and the note head is trained with mean squared error on the bar-local coordinates. The three task losses are combined via learned uncertainty weighting [23]:

$$\mathcal{L} = \frac{1}{2\sigma_s^2}\mathcal{L}_{\text{sys}} + \frac{1}{2\sigma_b^2}\mathcal{L}_{\text{bar}} + \frac{1}{2\sigma_n^2}\mathcal{L}_{\text{note}} + \log \sigma_s \sigma_b \sigma_n, \tag{3}$$

where σ_s, σ_b, and σ_n are learnable task-specific uncertainty parameters that balance the three objectives automatically.

3.6 Jump Detection and Recovery

Score discontinuities such as repeats, D.C., and coda jumps cause the active position to move non-monotonically. Rather than modeling each type separately, CODA treats all discontinuities as arbitrary position changes preceded by silence. This assumption is grounded in the observation of Nakamura et al. [5]. This also covers unscripted performer errors. CODA realizes this with two mechanisms demonstrated in Figure 3.

The jump-augmented training is illustrated in Figure 3(a). Jump augmentation splices each sample with a controllable probability: a short silence (3–12 frames) is inserted, followed by audio from a randomly selected destination on the same or a different page. For same-page jumps, the previous system and bar labels are frozen at the source position, forcing the model to relocalize from audio evidence against a biased temporal prior.

At inference time, the break mode monitors waveform energy using a hysteresis rule. As presented in Figure 3(b), when energy stays below a threshold for several consecutive frames, the tracker enters a break state: transition penalties are suppressed, the system beam is widened to cover all candidates, and the committed position is frozen. When energy rises again, penalties remain suppressed for a short grace window during which the tracker relocalizes to the jump destination. The Mamba hidden state is preserved throughout, maintaining temporal continuity across the silence gap.

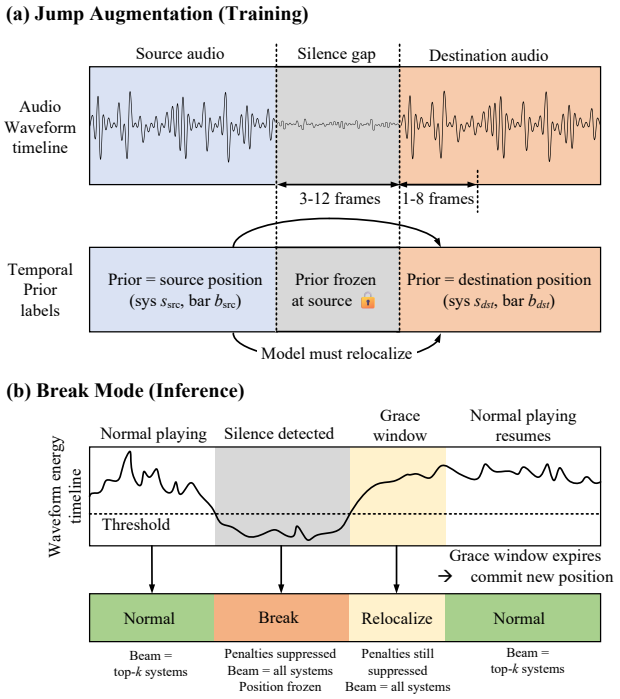

Figure 3: Jump recovery mechanism. (a) Jump augmentation during training. (b) Break mode at inference.

4. EXPERIMENTS AND RESULTS

4.1 Experimental Setup

4.1.1 Dataset.

Training and evaluation are both based on the MSMD dataset [24], using the preprocessed version provided by Henkel and Widmer [10], in which each piece is stored as a score image with per-note coordinate annotations and per-system and per-bar bounding box annotations, paired with a synthesized audio file (22,050 Hz). The standard split contains 354 training, 19 validation, and 94 test pieces.

4.1.2 Evaluation protocol.

We consider two evaluation settings matching [10]: Setting I uses the full 94-piece test split with synthetic score images and synthetic audio; Setting II pairs the same synthetic images with real piano recordings for a 16-piece subset [8], testing generalization to real audio. Settings III and IV of [10], which require commercially published scanned scores, are not publicly available and are therefore not included.

Following [10], we evaluate at each ground-truth note onset. Because image-based trackers predict positions in pixel space, we first convert each predicted position to the time domain by interpolating through the ground-truth onset-to-pixel mapping, then compute the absolute time difference between the predicted and true onset. We report the cumulative ratio of onsets tracked below five error thresholds: ≤ 0.05, 0.10, 0.50, 1.00, and 5.00 seconds. Higher ratios indicate better tracking. For methods that produce multi-resolution outputs, we additionally report *system accuracy* and *bar accuracy*, defined as the fraction of evaluated frames in which the predicted system or bar matches the ground-truth.

4.1.3 Baselines.

We compare against the image-based score following methods evaluated in [10]: **MM-Loc** [6], a supervised multimodal localization model; **RL** [8], a reinforcement-learning agent; **CUNet** [9], a conditional U-Net for full-page segmentation; **CYOLO** [25], the conditional YOLO detector predicting note-level bounding boxes; and **CYOLO-SB** [10], the multi-resolution variant that adds system and bar heads. All baseline numbers are taken directly from [10] under the same evaluation protocol. We also list **CYOLO-SB + A** for reference, noting that this variant uses additional proprietary training data (scanned scores and commercial recordings) that are not available to other methods. Methods that rely on automatic music transcription as a front end, such as Peter et al. [16], are not directly comparable because they depend on symbolic-level intermediate representations rather than operating on sheet images.

4.1.4 Implementation details.

Audio is processed at 22,050 Hz with a 2048-sample short-time Fourier transform (STFT) (hop size 1 102, ≈ 20 frames per second) and a 78-bin log-filterbank spanning 60 Hz–6 kHz. Score pages are converted to grayscale and scaled to a fixed width of 416 pixels. The Mamba audio encoder has two layers with hidden dimension 64, state dimension 16, convolution width 4, expansion factor 2, and projects to a 128-dimensional conditioning vector z_t. Both the system and bar selection heads use cross-attention with four heads over an audio buffer of 64 frames. The beam search retains k=3 system and m=3 bar candidates per frame. Break mode uses an energy onset threshold of 0.1, release threshold of 0.25, a silence onset of 3 frames (≈ 150 ms), and a grace window of 8 frames (≈ 400 ms). During break, priors are multiplied by zero and the beam covers all systems, with m=3 bars per system.

Training follows the two-phase curriculum described in Section 3. Phase 1 trains for 30 epochs with a learning rate of 5×10^{-4} under ground-truth system routing. Phase 2 fine-tunes for 20 epochs at 1×10^{-4} with scheduled sampling, where the probability of using the model's own system prediction ramps linearly to $p_{\max}$=0.7 over the first 5 epochs. Both phases use the AdamW optimizer with a batch size of 16, cosine learning-rate decay, and gradient clipping at norm 1.0. Data augmentation includes jump augmentation (Section 3.6), spatial shifts, tempo scaling, and cold-start truncation. Following [10], impulse response (IR) augmentation is also applied during training, convolving the audio with a randomly selected room impulse response on-the-fly to model varying microphone and room conditions. Jump augmentation samples destinations from a weighted mixture of six categories: *repeat* (40%), *bar correction* (15%), *skip* (15%), *restart* (10%), *page jump* (10%), and *random* (10%).

CODA has 2.0M trainable parameters. All training and experiments are conducted on a single NVIDIA RTX

A6000 GPU (48 GB) with an Intel Xeon Gold 5218R CPU and 64 GB RAM. At inference, CODA processes each audio frame in 12.8 ms (78.1 fps), within the 50 ms real-time budget at 20 fps.

4.2 Standard Tracking Results

Table 1 compares CODA to the baselines on the two evaluation settings described in Section 4.1. In Setting I, CODA achieves .914 at ≤ 0.10 s compared to .837 for CYOLO-SB, with bar accuracy improving from .890 to .975 and system accuracy from .963 to .991. In Setting II, CODA achieves .743 at ≤ 0.10 s versus .630 for CYOLO-SB.

Table 1: Score-following comparison. Each cell is the ratio of onsets tracked below the error threshold (higher is better); best in **red bold**, second best in blue underline. †Proprietary training data; baseline numbers from [10].

	Onset error threshold (s)					Frame accuracy	
Method	≤.05	≤.10	≤.50	≤1.0	≤5.0	Bar	Sys
Setting I: Synthetic images – Synthetic audio							
MM-Loc [6]	.707	.747	.839	.855	.917	–	–
RL [8]	.411	.435	.776	.856	.971	–	–
CUNet [9]	.726	.750	.855	.885	.937	–	–
CYOLO [25]	.830	.842	.885	.909	.984	–	–
CYOLO-SB [10]	.820	.837	.893	.912	.983	.890	.963
CYOLO-SB+A† [10]	.846	.861	.908	.927	.984	.892	.956
CODA (ours)	**.897**	**.914**	**.955**	**.981**	**.996**	**.975**	**.991**
Setting II: Synthetic images – Real audio recordings							
MM-Loc [6]	.364	.406	.585	.611	.735	–	–
RL [8]	.185	.200	.476	.603	.901	–	–
CUNet [9]	.113	.125	.224	.266	.443	–	–
CYOLO [25]	.563	.581	.712	.749	.919	–	–
CYOLO-SB [10]	.610	.630	.799	.832	.960	.829	.917
CYOLO-SB+A† [10]	.682	.706	.865	.891	.981	.865	.941
CODA (ours)	**.721**	**.743**	**.883**	**.914**	**.987**	**.891**	**.953**

4.3 Jump Recovery Evaluation

To fairly evaluate jump recovery, we construct a jump test benchmark from the 94 MSMD test pieces. We manually annotated the repeat structure of each piece (repeat barlines, da capo, volta brackets, binary form, etc.). Of the 94 pieces, 66 contain written repeat structures and 28 do not. The benchmark is partitioned into two subsets accordingly. The *repeat subset* comprises the 66 pieces with repeats, producing 131 jumps that follow the annotated performance order. The *random subset* comprises the remaining 28 pieces without repeats, each receiving 3 randomly placed jumps (84 jumps in total) as a stress test.

We report three metrics on post-jump segments: *system recovery rate* (fraction of jumps where the correct system is identified within 1.0 and 2.0 s), *mean recovery latency* (time in seconds from audio resumption to the first correct system prediction), and *post-jump tracking accuracy* (≤ 1.0 s threshold in the 5 s window after each jump).

Table 2 shows the results. Without a jump-handling mechanism, CYOLO-SB yields low recovery rates across both subsets. CODA without break mode also struggles because learned transition penalties resist large position changes. On the repeat subset, CODA recovers the correct system within 1 s in 78% of jumps. Recovery is higher on the repeat subset than the random subset (.78 vs. .64 at 1 s).

Table 2: Jump recovery results on the repeat-aware MSMD test set. **Rec.@k s**: fraction of jumps with correct system within k s. **Lat.**: mean seconds to first correct system prediction. **Post-J.**: fraction of onsets tracked within ≤ 1.0 s in the 5 s window after each jump.

		Sys recovery			Post-jump
Subset	Method	@1 s	@2 s	Lat. (s)	≤1 s
Repeat	CYOLO-SB	.12	.20	4.31	.35
	CODA w/o break	.29	.44	2.63	.54
	CODA (full)	**.78**	**.91**	**0.72**	**.82**
Random	CYOLO-SB	.08	.15	5.18	.27
	CODA w/o break	.23	.38	3.07	.46
	CODA (full)	**.64**	**.80**	**1.24**	**.71**

4.4 Ablation Study

To isolate the contribution of each component, we evaluate five ablated variants of CODA on Setting I (synthetic images and audio). Each variant removes or disables a single component while keeping all others unchanged.

Table 3 shows the results. The largest degradation comes from removing the cascade ($-.045$ at ≤ 0.10 s, bar accuracy drops from .975 to .931). Beam search, cross-attention, and temporal priors contribute the next-largest gains ($-.026$, $-.019$, and $-.014$ at ≤ 0.10 s, respectively). Scheduled sampling has the smallest individual effect, but consistently improves all thresholds.

Table 3: Ablation study on the MSMD synthetic test set (Setting I). Each row removes one component from the full model.

	Onset error threshold (s)				Frame accuracy	
Variant	≤.05	≤.10	≤.50	≤1.0	Bar	Sys
CODA (full)	**.897**	**.914**	**.955**	**.981**	**.975**	**.991**
w/o cascade	.851	.869	.923	.952	.931	.967
w/o cross-attention	.878	.895	.943	.972	.958	.985
w/o beam search	.870	.888	.938	.968	.949	.982
w/o temporal priors	.882	.900	.948	.975	.963	.988
w/o sched. sampling	.888	.906	.951	.978	.968	.989

5. DISCUSSION AND CONCLUSION

This paper presents CODA, which formulates image-based score following as cascaded selection over known candidates and introduces a silence-driven break mode for jump recovery. We also contribute a repeat-aware jump test benchmark with manually annotated repeat structures for the MSMD test set, providing the first standardized evaluation protocol for discontinuity handling in image-based score following. Despite achieving state-of-the-art performance, several limitations remain. CODA is trained and evaluated exclusively on solo piano from the MSMD dataset [24]. Larger datasets that pair real recordings with scanned scores would benefit not only this work but the field as a whole. Extending CODA to other solo instruments is a natural first step. Beyond solo, chamber music and orchestral settings pose additional challenges due to timbral overlap, denser layouts, and multiple simultaneous parts. Finally, evaluation under real-world conditions, including scanned pages, commercial recordings, and live microphone input, is needed to further assess the method.

6. REFERENCES

[1] N. Orio, S. Lemouton, and D. Schwarz, "Score following: State of the art and new developments," in *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*, 2003, pp. 36–41.

[2] R. B. Dannenberg and C. Raphael, "Music score alignment and computer accompaniment," *Communications of the ACM*, vol. 49, no. 8, pp. 38–43, 2006.

[3] S. Dixon, "An on-line time warping algorithm for tracking musical performances," in *IJCAI*, 2005, pp. 1727–1728.

[4] A. Cont, "A coupled duration-focused architecture for real-time music-to-score alignment," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 32, no. 6, pp. 974–987, 2010.

[5] T. Nakamura, E. Nakamura, and S. Sagayama, "Real-time audio-to-score alignment of music performances containing errors and arbitrary repeats and skips," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 24, no. 2, pp. 329–339, 2016.

[6] M. Dorfer, A. Arzt, and G. Widmer, "Towards score following in sheet music images," in *Proceedings of the 17th International Society for Music Information Retrieval Conference (ISMIR)*, 2016, pp. 789–795.

[7] M. Dorfer, F. Henkel, and G. Widmer, "Learning to listen, read, and follow: Score following as a reinforcement learning game," in *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, 2018, pp. 784–791.

[8] F. Henkel, S. Balke, M. Dorfer, and G. Widmer, "Score following as a multi-modal reinforcement learning problem," *Transactions of the International Society for Music Information Retrieval*, vol. 2, no. 1, pp. 67–81, 2019.

[9] F. Henkel, R. Kelz, and G. Widmer, "Learning to read and follow music in complete score sheet images," in *Proceedings of the 21st International Society for Music Information Retrieval Conference (ISMIR)*, 2020, pp. 780–787.

[10] F. Henkel and G. Widmer, "Real-time music following in score sheet images via multi-resolution prediction," *Frontiers in Computer Science*, vol. 3, p. 718340, 2021.

[11] A. Morsi and X. Serra, "Bottlenecks and solutions for audio to score alignment research," in *Proceedings of the 23rd International Society for Music Information Retrieval Conference (ISMIR)*, 2022, pp. 272–279.

[12] J. Park, C. E. Cancino-Chacón, S. Chiruthapudi, and J. Nam, "Matchmaker: An open-source library for real-time piano score following and systematic evaluation," in *Proceedings of the 26th International Society for Music Information Retrieval Conference (ISMIR)*, 2025, pp. 91–99.

[13] A. Arzt and G. Widmer, "Towards effective 'any-time' music tracking," in *Proceedings of the Fifth Starting AI Researchers' Symposium (STAIRS)*, ser. Frontiers in Artificial Intelligence and Applications, vol. 222. IOS Press, 2010, pp. 24–36.

[14] F. Korzeniowski, F. Krebs, A. Arzt, and G. Widmer, "Tracking rests and tempo changes: Improved score following with particle filters," in *ICMC*, 2013.

[15] Y. Jiang and C. Raphael, "Score following with hidden tempo using a switching state-space model," in *Proceedings of the 21st International Society for Music Information Retrieval Conference (ISMIR)*, 2020, pp. 693–699.

[16] S. Peter, P. Hu, and G. Widmer, "Pairing real-time piano transcription with symbol-level tracking for precise and robust score following," in *Proceedings of the Sound and Music Computing Conference (SMC)*, 2025.

[17] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.

[18] M. Shan and T. J. Tsai, "Improved handling of repeats and jumps in audio–sheet image synchronization," in *Proceedings of the 21st International Society for Music Information Retrieval Conference (ISMIR)*, 2020, pp. 62–69.

[19] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," in *Conference on Language Modeling (COLM)*, 2024.

[20] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask r-cnn," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2961–2969.

[21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[22] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks," in *Advances in Neural Information Processing Systems*, vol. 28, 2015, pp. 1171–1179.

[23] A. Kendall, Y. Gal, and R. Cipolla, "Multi-task learning using uncertainty to weigh losses for scene geometry and semantics," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 7482–7491.

[24] M. Dorfer, J. Hajič, jr., A. Arzt, H. Frostel, and G. Widmer, "Learning audio–sheet music correspondences for cross-modal retrieval and piece identification," *Transactions of the International Society for Music Information Retrieval*, vol. 1, no. 1, pp. 22–33, 2018.

[25] F. Henkel and G. Widmer, "Multi-modal conditional bounding box regression for music score following," in *2021 29th European Signal Processing Conference (EUSIPCO)*. IEEE, 2021, pp. 356–360.